

# VALBOT API

# VALBOT API

Auditoria automática de exames práticos DETRAN (Res. CONTRAN 1.020/2025).

- **Base URL:** <https://valbot.com.br>
- **Swagger UI:** <https://valbot.com.br/api/docs>
- **ReDoc:** <https://valbot.com.br/api/redoc>
- **OpenAPI JSON:** <https://valbot.com.br/api/openapi.json>

## Autenticação

| Endpoint                                                                                           | Auth                                    | Visibilidade           |
|----------------------------------------------------------------------------------------------------|-----------------------------------------|------------------------|
| POST /api/exams/init-upload                                                                        | X-API-Key (scope exams:create )         | público                |
| POST /api/exams/{id}/finalize                                                                      | X-API-Key (scope exams:create )         | público                |
| GET /api/health                                                                                    | nenhuma                                 | público (health check) |
| GET /api/docs , /api/redoc , /api/openapi.json                                                     | nenhuma                                 | público (docs)         |
| POST /api/admin/api-keys                                                                           | X-Admin-Token (env VALBOT_ADMIN_TOKEN ) | admin                  |
| Todos os demais ( /api/exams , /api/videos , /api/laudo/* , /api/bench/* , /api/analyses/* , etc.) | X-Admin-Token                           | interno                |

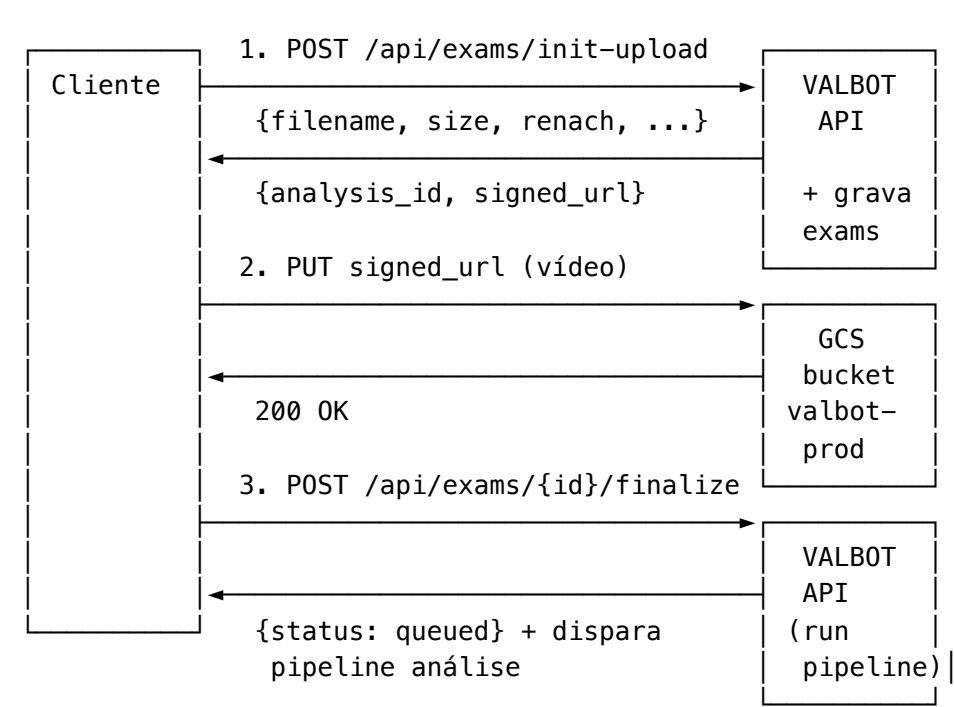
### Como usar a key:

X-API-Key: vbk\_live\_<32\_bytes\_hex>

Sem header → 401 X-API-Key header obrigatório . Key revogada / scope errado → 401 API key inválida, revogada ou sem scope .

last\_used\_at é atualizado a cada chamada bem-sucedida.

## Fluxo de upload (3 passos)



Vídeo nunca passa pela VM da API — sobe direto pro GCS via signed URL (expira em 20 min).

### POST /api/exams/init-upload

Cria registro pendente em `exams` e devolve signed PUT URL pro GCS.

**Headers:** - `X-API-Key: vbk_live_<...>` (obrigatório, scope `exams:create` ) - `Content-Type: application/json`

**Body (JSON):**

| Campo        | Tipo   | Obrigatório | Notas                                     |
|--------------|--------|-------------|-------------------------------------------|
| filename     | string | sim         | nome do arquivo<br>( .mp4 / .mov / .m4v ) |
| size_bytes   | int    | sim         | tamanho do vídeo em bytes (max 600 MB)    |
| content_type | string | sim         | video/mp4 ,<br>video/quicktime etc        |
| renach       | string | sim         | RENACH/CNH do candidato                   |

| Campo                | Tipo   | Obrigatório | Notas                          |
|----------------------|--------|-------------|--------------------------------|
| candidato_nome       | string | sim         |                                |
| candidato_cpf        | string | sim         |                                |
| processo             | string | sim         | nº processo DETRAN             |
| categoria            | string | sim         | A , B , AB , C , D , E         |
| veiculo              | string | sim         | modelo/placa                   |
| local                | string | sim         | unidade DETRAN / CFC           |
| examinador           | string | sim         |                                |
| auto_escola          | string | sim         |                                |
| rubrica              | string | sim         | 1020/2025 (default)            |
| training_annotations | string | não         | JSON livre p/ treino do modelo |
| sha256_client        | string | não         | SHA-256 client-side p/ cache   |

#### Resposta 200 :

```
{
  "analysis_id": "5d4a28262e0644409509be84666bc330",
  "signed_url": "https://storage.googleapis.com/valbot-prod/uploads/.../video.mp4?X-Goog-...",
  "gs_path": "gs://valbot-prod/uploads/5d4a.../video.mp4",
  "expires_in_minutes": 20,
  "expected_content_type": "video/mp4"
}
```

#### Resposta 401 :

```
{"detail": "X-API-Key header obrigatório"}
```

#### Exemplo curl :

```
curl -X POST https://valbot.com.br/api/exams/init-upload \
-H "X-API-Key: vbk_live_<sua_key>" \
-H "Content-Type: application/json" \
-d '{
  "filename": "exame-001.mp4",
  "size_bytes": 52428800,
  "content_type": "video/mp4",
}
```

```
"renach": "SP-12345678",
"candidato_nome": "Joao Silva",
"candidato_cpf": "12345678901",
"processo": "PROC-2026-001",
"categoria": "B",
"veiculo": "VW Gol ABC1D23",
"local": "DETRAN Centro",
"examinador": "Maria Souza",
"auto_escola": "CFC Estrela",
"rubrica": "1020/2025",
"training_annotations": ""
}'
```

---

## PUT <signed\_url> (browser/cliente → GCS direto)

```
curl -X PUT "<signed_url>" \
-H "Content-Type: video/mp4" \
--data-binary @exame-001.mp4
```

Sem auth header (a signed URL já carrega assinatura). Retorna 200 OK quando o blob entra no bucket.

---

## POST /api/exams/{analysis\_id}/finalize

Confirma chegada do blob no GCS e dispara análise em background.

**Headers:** Content-Type: application/json **Body:** {"sha256\_client": "<opcional>"}

**Resposta 200 :**

```
{"analysis_id": "5d4a...", "status": "queued", "gs_path": "gs://valbot-prod/.../video.mp4"}
```

Idempotente — se já está queued / running / processed , retorna o mesmo payload com idempotent: true .

---

## GET /api/exams/{analysis\_id}

Status + metadados.

```
{
  "analysis_id": "...",
  "status": "queued|running|processed|failed",
  "candidato": {...},
```

```
"exam": {...},  
"result_url": "/api/exams/.../result"  
}
```

## GET /api/exams/{analysis\_id}/result

Laudo completo em JSON quando `status == processed`.

## POST /api/exams/{analysis\_id}/reanalyze

Re-roda pipeline (mesmo vídeo, novo run).

## GET /api/laudo/{hash}/pdf

PDF do laudo.

---

## Endpoints internos (X-Admin-Token obrigatório)

Todos exigem header `X-Admin-Token: <env VALBOT_ADMIN_TOKEN>`. Sem header → 401 X-Admin-Token requerido (endpoint interno).

| Endpoint                               | O que faz                     |
|----------------------------------------|-------------------------------|
| GET /api/exams                         | lista exames                  |
| GET /api/exams/{id}                    | status + metadados            |
| GET /api/exams/{id}/result             | laudo JSON quando processed   |
| POST /api/exams/{id}/reanalyze         | re-roda pipeline              |
| GET /api/laudo/{hash}/pdf              | PDF do laudo                  |
| GET /api/laudo-atual                   | laudo da sessão               |
| GET /api/videos                        | lista vídeos do storage local |
| GET /api/rubricas/{slug}               | metadados da rubrica          |
| GET /api/bench/videos                  | vídeos do bench               |
| GET /api/bench/{slug}/version          | versão do bench               |
| GET /api/bench/{slug}/models           | modelos do bench              |
| GET /api/bench/{slug}/laudo/{model_id} | laudo do bench                |

| Endpoint                             | O que faz          |
|--------------------------------------|--------------------|
| GET /api/analyses/hash/{hash}/result | resultado por hash |
| GET /api/analyses/{hash}/annotations | anotações          |
| GET /preset/{version}/{variant}      | preset VLM         |

## POST /api/admin/api-keys (admin)

Cria nova API key. Plaintext mostrada **uma única vez**.

**Headers:** - X-Admin-Token: <env VALBOT\_ADMIN\_TOKEN> - Content-Type: application/json

**Body:**

```
{"name": "techpark", "scopes": ["exams:create"]}
```

**Resposta:**

```
{
  "id": "uuid",
  "name": "techpark",
  "scopes": ["exams:create"],
  "key": "vbk_live_<64-hex>",
  "warning": "Esta key não será mostrada novamente. Salve agora."
}
```

**Revogar key:** UPDATE api\_keys SET revoked\_at = NOW() WHERE name = 'techpark'; no Postgres.

## Scopes disponíveis

| Scope        | Permite                     |
|--------------|-----------------------------|
| exams:create | POST /api/exams/init-upload |

(novos scopes serão adicionados conforme endpoints forem fechados)

## Códigos HTTP

| Código | Significado |
|--------|-------------|
| 200    | OK          |

| Código | Significado                                        |
|--------|----------------------------------------------------|
| 401    | header faltando ou key inválida/revogada/sem scope |
| 404    | recurso não encontrado                             |
| 415    | content_type não suportado                         |
| 422    | tamanho declarado $\neq$ tamanho do blob           |
| 503    | infra indisponível (signed URL, etc)               |

## Persistência

Toda chamada `init-upload` grava em `exams` (Postgres): - `id` UUID - `hash` (= `analysis_id`) - `renach`, `candidato_*`, `processo`, `categoria`, `veiculo`, `local_unidade`, `examinador`, `auto_escola`, `rubrica` - `gs_video` (`gs://valbot-prod/uploads/.../video.mp4`) - `status` (`queued` / `running` / `processed` / `failed`) - `created_at`, `updated_at`

Vídeo vive em GCS (`gs://valbot-prod/uploads/`), durabilidade 11 noves. Postgres em disco persistente independente da VM. Backup diário 03:00 UTC → `gs://valbot-prod/backups/pg/YYYY-MM-DD.sql.gz` (retenção 30d).